Table of Contents

1. Rationale 1

1.1 Problem Description 1
1.2 Background 1
1.3 Community Impact 1
1.4 Proposed Solution 2
1.5 Computational Thinking Process 2
1.6 Algorithm Design 3
1.7 SDG Alignment 3
1.8 Open Source Justification 3
1.9 Digital Public Goods Justification 3

2. Algorithm 5

3. Pseudocode 7

4. Flowcharts 15

5. Functional Modules 23

5.1 Modules 23
5.2 Functions 24
5.3 Variables, Constants and Data Types 25
5.4 Decision Structures 26
5.5 Repetition Structures 27
5.6 Boolean Logic 27
5.7 Input Validation and Error Handling 28

6. Expected Outputs 30

6.1 Sample Data 30
6.2 Test Scenarios 30
6.3 Fare Classification Tests 35
6.4 Error-Handling Tests 36

7. GitHub Evidence 37

8. References 39

1. Rationale

1.1 Problem Description

Moving around Freetown is a daily struggle. The city has about one million residents, and for most of them public transport means poda-podas, shared taxis, okadas and kekes run by private operators. A World Bank diagnostic found that these four modes carried about 92% of passenger demand, with poda-podas alone carrying about 30% (World Bank, 2024).

Congestion is the problem people talk about most, but a software system cannot fix congestion on its own. This project focuses on a narrower problem inside it, one that logic and data can actually address: **commuters have no reliable way to know the official fare for a trip, to check whether they have been overcharged, or to report it, and the authorities have no organised record of where these problems happen.**

1.2 Background

Why the problem exists. Public transport fares in Sierra Leone are set by government and change whenever fuel prices change. In 2026 alone, petrol rose from NLe 25.00 to NLe 28.50 per litre on 28 January and to NLe 32.00 in March (BudgIT Sierra Leone, 2026), reached NLe 35.00, and was cut to NLe 33.00 on 29 June (Sierraloaded, 2026a). Each change brings a new fare list. The latest, effective 30 June 2026, cut fares by 12.5%, for example from NLe 7.00 to NLe 6.10 between Lumley and Regent Road (Sierraloaded, 2026b). These lists are shared through press releases, radio and social media, yet only about 20.8% of the population uses the internet, while mobile connections equal about 101% of the population (DataReportal, 2026). Many commuters therefore hear about new fares late or not at all, and drivers fill the gap with their own prices.

Overcharging is not new. Commuters have described drivers cutting routes into short sections and charging "two-way" fares (Awoko, 2013), and taxi drivers charging two to five times the normal fare (Awoko, 2015). Drivers point to daily targets set by vehicle owners, time lost in traffic, and payments demanded at checkpoints. Whatever the cause, a passenger at a crowded junction during rush hour has no bargaining power and no simple way to complain.

Government has recognised the cost of the problem. When launching its urban mobility project, the President said, "There is a heavy direct and indirect cost to congestion" (State House, 2019). The USD 52 million Integrated and Resilient Urban Mobility Project (IRUMP) is funded by the Government of Sierra Leone and the World Bank (State House, 2024). The Public Transport Act 2023 created the Sierra Leone Public Transport Authority (SLPTA), and Metro Transport Company Limited started "Waka Fine" bus services with 50 buses, while average speeds rose from 11 km/h to 15 km/h (World Bank, 2024). Still, most trips are made in informal vehicles, okadas and kekes remain a major safety concern (Greene and Morgan, 2024), and road crashes kill more than 16 people per 100,000 each year (World Bank, 2024).

Who is affected. Daily commuters such as workers, traders, market women carrying goods, school children, and university students like us; persons with disabilities and older people, who cannot run after vehicles or argue at a junction; honest drivers, who lose passengers' trust because of the ones who cheat; and regulators such as the SLPTA and the Sierra Leone Road Safety Authority, who make decisions without reliable data from the street.

1.3 Community Impact

Problem being solved: the gap between official fares and what passengers actually pay, and the lack of a simple reporting channel. **Expected impact:** a commuter can confirm the official fare in about a minute from any phone, overcharging becomes visible and measurable, and enforcement teams can be sent to the routes with the most complaints instead of guessing. **Beneficiaries:** commuters first, then

honest operators, then the SLPTA, the Ministry of Transport and Aviation and the Sierra Leone Police, who gain a clear picture of problem routes. **Long-term sustainability:** the system is cheap to run because it uses USSD and SMS rather than an app; an authorised officer updates every fare in one step after each announcement; and because the design is open source, local developers and universities can maintain and extend it without depending on one company.

1.4 Proposed Solution

The proposed system, **SaloneMove**, is a fare check and reporting system reached through a USSD short code, with SMS replies, so it works on any mobile phone without data. Officers use the same menu and log in with a PIN. It offers four services, each designed as a module:

- **Phone Validation and Verification:** two separate steps. *Validation* checks that the caller has typed a correctly formed Sierra Leone mobile number (9 digits starting with a valid mobile prefix), allowing three attempts. *Verification* then texts a one-time code to that number and asks the caller to type it back, which proves they actually hold the phone; the code allows three tries and expires after five minutes.
- **Check a Fare:** gives the official fare for a route and vehicle, and says whether the amount paid was underpaid, correct, overcharged or a serious overcharge.
- **Report a Problem:** offered after every fare check; records overcharging, short-routing, reckless driving, harassment or overloading against that route, and sends an SMS reference number. An overcharging report is accepted only when the fare check found an overcharge.
- **Officer Portal:** protected by a PIN that locks after three wrong tries; lists hotspot routes with many open complaints, and applies a government fare change to every fare at once.

Every number, menu choice and place name is checked before it is used, and a clear message says what is wrong and asks again, so a mistyped entry never stops the system or saves bad data.

How it addresses the problem: each cause in Section 1.2 has a matching feature. Late fare news is answered by a fare lookup on any phone, the inability to check a fare by the fare check, the missing complaint channel by reports with reference numbers, the lack of data by the hotspot report, and frequent fare changes by a one-step fare update. Because every report comes from a verified number, the data officers rely on is harder to fake. SaloneMove does not remove traffic, but it makes fares transparent and gives passengers a voice.

1.5 Computational Thinking Process

- **Decomposition:** the problem was broken into parts that can be solved separately: checking who is calling, checking the fare, reporting, and giving officers usable data. Each part became a module, and repeated jobs (searching, classifying a fare, reading and checking an entry) became functions.
- **Pattern recognition:** every fare question has the same shape (route, vehicle, fare), so one record type and one search function serve every check; every report has one shape, so one counting routine produces the hotspot report; and every entry follows one pattern: read it as text, check it, and ask again with a message until it is valid. That pattern became three small input functions.
- **Abstraction:** details that do not change the logic were left out. The system needs the fare list, the complaints and the caller's verified number, not GPS or driver profiles.
- **Algorithm design:** each module was written as steps with decisions and loops, then as pseudocode and flowcharts (Sections 2 to 4), and desk-checked with test data, including wrong and boundary entries (Section 6).

1.6 Algorithm Design

The design is top-down and modular, with six modules and eight functions. The main module validates and then verifies the caller's phone number, then loops through a menu until the user exits. Each function returns one value: FindFare() returns the position of the matching fare record (or −1), so one search gives both the fare and the route name; ClassifyFare() returns one of four fare statuses; VerifyPhone() returns whether the one-time code was confirmed; and the three input functions return only checked values. Every entry is read as text and checked before it is converted or used. A linear search suits the fare table because it is small and changes often, and route names are compared in capital letters so the way a commuter types a place does not matter. Nested loops count open complaints per route. Every report is linked to a route from the fare table, so complaints can be counted reliably.

1.7 SDG Alignment

SaloneMove supports three of the Sustainable Development Goals set out in the 2030 Agenda (United Nations, 2015):

SDG and Target	How SaloneMove contributes
SDG 11 Sustainable Cities, Target 11.2: safe, affordable, accessible transport	Makes official fares known to everyone on any phone, protects vulnerable passengers from overcharging, and supports the shift to organised public transport.
SDG 3 Good Health, Target 3.6: fewer road traffic deaths and injuries	Reckless driving and overloading reports identify dangerous routes and vehicles for enforcement.
SDG 16 Strong Institutions, Target 16.6: accountable, transparent institutions	Every complaint gets a reference number and is counted, so regulators can be measured on how they respond.

1.8 Open Source Justification

Why open source: SaloneMove handles public fare data and public complaints, so the logic that decides what counts as "overcharged" should be open for anyone to inspect. **Benefits for Sierra Leone:** no licence fees for government; no lock-in to a single vendor; the same design can be adapted for Bo, Kenema and Makeni or for inter-district routes; and local developers gain experience on a real national system. **Potential community contributions:** students and developers can fix bugs and add features; transport unions can check route names and fares against the official list; volunteers can translate the menus into Krio, Temne and Mende; and researchers can study the anonymised complaint data. The code would be released under the MIT licence and the documentation under Creative Commons CC BY 4.0.

1.9 Digital Public Goods Justification

A Digital Public Good is open source software, data or content that meets the nine indicators of the Digital Public Goods Standard (Digital Public Goods Alliance, n.d.). SaloneMove is designed to qualify, and to be **accessible** (any phone, no data, local languages), **open** (code, logic and fare data are public) and of **public benefit** (free to commuters and reusable by any city with a similar transport system).

Indicator	How it is met	Indicator	How it is met
1. SDG relevance	Supports SDG 11.2, 3.6 and 16.6	6. Non-PII data extraction	Fare tables and anonymised complaint counts exported as open data

Indicator	How it is met	Indicator	How it is met
2. Open licensing	MIT (code), CC BY 4.0 (documents)	7. Privacy and applicable laws	Only the verified phone number is stored; it is never published or exported. One-time codes are never stored
3. Clear ownership	A public body (proposed: SLPTA)	8. Standards and best practices	Modular design, Git version control, validation of every entry
4. Platform independence	Open standards (USSD, SMS)	9. Do no harm (9A to 9C)	Hashed officer PIN that locks after 3 wrong tries; reports only from verified numbers; reports kept private; abusive reports closed
5. Documentation	README, pseudocode, flowcharts and report published with the code		

2. Algorithm

The algorithm below describes, in plain English, what SaloneMove does from the moment a user dials the short code. Each part matches a module in the pseudocode (Section 3) and a flowchart (Section 4).

Main Program

1. Start. Set attempts to 0 and isValid to FALSE, and display "Welcome to SaloneMove".
2. **Validation.** While attempts is less than 3 and isValid is FALSE: read the phone number. If it has 9 digits and starts with a valid mobile prefix (for example 076 or 088), set isValid to TRUE; otherwise add 1 to attempts and display how many attempts are left.
3. If isValid is still FALSE, display "Too many invalid attempts. Session ended." and stop.
4. **Verification.** Otherwise create a random six-digit code and send it to the number by SMS. If the SMS cannot be sent, display "Code could not be sent. Please try again later." If it is sent, allow up to 3 tries: a try succeeds only if the code typed matches and no more than 5 minutes have passed since it was sent; each failed try displays "Wrong or expired code." with the attempts left. If the number is not verified, display "Phone number not verified. Session ended." and stop.
5. Otherwise set running to TRUE. While running is TRUE, display the menu (1 Check a Fare / Report a Problem, 2 Officer Login, 0 Exit) and read the choice with ReadChoice, which accepts only 0, 1 or 2.
6. If the choice is 1, perform Check a Fare. If 2, open the Officer Portal. If 0, set running to FALSE.
7. When the loop ends, display "Thank you for using SaloneMove. Waka fine!" and stop.

Check a Fare

1. Read the starting point and the destination with ReadPlace.
2. Show the vehicle menu (1 Poda-poda, 2 Keke, 3 Okada) and read the choice with ReadChoice (1 to 3).
3. Search the fare table from the first record. A record matches when its vehicle type is the one chosen and its route is the two places in either order, compared in capital letters. Stop at the first match; if there is none, the result is −1.
4. If the result is −1, display "Route not found. Check the spelling or try another vehicle." If the record's official fare is 0 or less (a damaged record), display "Fare record error. Please try again later.", so the percentage is never divided by zero. In both cases return to the menu.
5. Otherwise take the official fare and the route name from the matching record and set the status to NOT CHECKED. Show the fare and read the amount paid with ReadNumber, which accepts only a number from 0 (skip) to 10 times the official fare.
6. If an amount was entered, calculate difference = amount paid − official fare, rounded to 2 decimal places, and percent = difference ÷ official fare × 100. If the difference is more than NLe 0.50 below the fare, the payment is **UNDERPAID**; else if it is within NLe 0.50 either way, it is **CORRECT**; else if percent is 50 or more, it is a **SERIOUS OVERCHARGE**; otherwise it is **OVERCHARGED**. Display "Correct fare.", the amount underpaid with a request to pay the official fare, or the overcharge status with the amount and percentage.
7. Ask whether the user wants to report a problem with this vehicle and read the answer with ReadChoice (1 or 2). If the answer is 1 (Yes), perform Report a Problem with the route, the amount paid and the fare status.

Report a Problem

1. Show the five problem types and read the choice with ReadChoice (1 to 5).
2. If the choice is Overcharging but the fare status is not OVERCHARGED or SERIOUS OVERCHARGE, display "Report refused: the fare check did not find an overcharge." and return.

3. Otherwise read the plate number. While it is not 0 and not valid (5 to 8 letters and digits once spaces are removed), display "Invalid plate number" and read it again. 0 means the plate was not seen.
4. Add 1 to reportCounter and build the reference number in the form SLM-YEAR-NNNNNN.
5. Save the complaint with the reference, phone number, plate (in capitals, without spaces), route, problem type, amount, fare status, date and time, and status OPEN.
6. Send an SMS with the reference number; if it cannot be sent, display "SMS not sent. Please write down your reference." Display the reference on screen.

Officer Portal

1. If the caller's number is on the locked list, display "Officer login is locked for this number." and return.
2. Otherwise allow up to 3 PIN attempts. A PIN is accepted only if its hash matches the stored hash; each wrong PIN displays the attempts left. After 3 wrong PINs, add the number to the locked list and display "Access denied. Officer login locked for this number."
3. If the PIN is accepted, until the officer chooses 0, display the officer menu (1 Hotspot Report, 2 Apply Fare Change, 0 Back), read the choice with ReadChoice and carry it out.
4. **Hotspot Report:** for each route, count the complaints on that route that are still OPEN. If the count is 3 or more, display the route as a hotspot and add 1 to hotspotCount. At the end, display the total, or "No hotspots this period." if there are none.
5. **Apply Fare Change:** read the percentage with ReadNumber, which accepts only a number from −50 to 50. If it is 0, display "Rejected: the change cannot be 0%." Otherwise ask for confirmation (ReadChoice, 1 or 2). If confirmed, multiply every fare by (1 + percent ÷ 100), round to 2 decimal places and display how many fares were updated; if not, display "No changes made."

Input Functions

Every number, menu choice and place name is read as text and checked before it is converted or used. If it is wrong, a message says what is expected and the user types it again. The phone number, verification code and officer PIN are different: they allow only 3 attempts, because repeated guessing there is a security risk.

1. **ReadChoice(low, high):** accepts only digits whose value is from low to high, and returns the number.
2. **ReadNumber(low, high):** accepts only a number from low to high, and returns it rounded to 2 decimal places.
3. **ReadPlace(prompt):** shows the prompt and accepts only letters and spaces, with at least 2 letters, and returns the place without spaces at either end.

3. Pseudocode

The pseudocode uses standard conventions: keywords in capitals, ← for assignment, = for comparison, ≠ for "not equal", and indentation to show structure. The built-in routines it relies on are listed at the top. It is organised into the same four parts as the flowcharts, and every statement appears in the matching flowchart. Every INPUT reads text into a STRING variable, and AND and OR stop as soon as the result is known, so TO_INTEGER() and TO_REAL() only ever receive text that has already passed IsAllDigits() or IsNumber().

```
PROGRAM SaloneMove
// Commuter Fare Check and Reporting System for Freetown
// PROG101 Principles of Programming Logic and Design

// Conventions: ← assigns a value, = compares, ≠ means "not equal".
// Arrays start at index 0. DISPLAY shows REAL values to 2 decimal places.
// Every INPUT arrives as text. AND and OR stop as soon as the result is known.
// Built-in routines: LENGTH(x), SUBSTRING(text, start, count), UPPERCASE(text),
// TRIM(text) (removes spaces at either end), REMOVE_SPACES(text), IsAllDigits(text),
// IsNumber(text) (TRUE for 10, 6.10 or -3), IsLettersAndDigits(text),
// IsLettersAndSpaces(text), TO_INTEGER(text), TO_REAL(text), CONTAINS(list, value),
// ROUND(number, places) (0 places gives a whole number), ABS(number),
// PAD_LEFT(number, width, character), RANDOM_INTEGER(low, high), YEAR(dateTime)
// (the year as text), SECONDS_BETWEEN(start, end), GetCurrentDateTime(),
// HASH(text) (one-way, so PINs are never stored).
// SendSMS(phone, message) is provided by the mobile network's SMS gateway;
// it returns TRUE if the gateway accepted the message.

// ======================== CONSTANTS ========================
CONSTANT MAX_LOGIN_ATTEMPTS = 3 // INTEGER, tries for the phone number, code and PIN
CONSTANT PHONE_LENGTH = 9 // INTEGER, e.g. 076123456
CONSTANT CODE_EXPIRY_SECONDS = 300 // INTEGER, a one-time code lasts 5 minutes
CONSTANT MIN_PLACE_LENGTH = 2 // INTEGER, shortest place name accepted
CONSTANT FARE_TOLERANCE = 0.50 // REAL, NLe allowed either way for lack of small change
CONSTANT SERIOUS_PERCENT = 50 // REAL, overcharge % that counts as serious
CONSTANT MAX_AMOUNT_FACTOR = 10 // REAL, amount paid may be at most 10 × the fare
CONSTANT HOTSPOT_THRESHOLD = 3 // INTEGER, open complaints per route
CONSTANT MAX_FARE_CHANGE = 50 // REAL, largest % change accepted

// ========================= RECORDS =========================
RECORD FareRecord
    route : STRING // e.g. "Lumley - Regent Road"
    vehicleType : STRING // "PODA", "KEKE" or "OKADA"
    officialFare : REAL // in New Leones (NLe)
END RECORD

RECORD Complaint
    referenceID : STRING
    phoneNumber : STRING // verified number of the person reporting
    plateNumber : STRING // "0" if the plate was not seen
    route : STRING
    category : STRING
    amountCharged : REAL
    fareStatus : STRING // result of the fare check, e.g. "OVERCHARGED"
    dateTime : DATETIME
    status : STRING // "OPEN" or "CLOSED"
```

```
END RECORD

// ======================== GLOBAL DATA ========================
// fareTable, routeList, complaints, validPrefixes, lockedNumbers,
// officerPINHash and reportCounter are loaded from the database at start-up.
DECLARE fareTable : ARRAY OF FareRecord
DECLARE routeList : ARRAY OF STRING // every route in fareTable, listed once
DECLARE complaints : ARRAY OF Complaint
DECLARE validPrefixes : ARRAY OF STRING // mobile prefixes, e.g. "076", "088"
DECLARE lockedNumbers : ARRAY OF STRING // numbers locked out of the officer portal
DECLARE officerPINHash : STRING
DECLARE reportCounter : INTEGER // number of reports issued so far
DECLARE vehicleTypes : ARRAY OF STRING
DECLARE categories : ARRAY OF STRING
vehicleTypes ← ["PODA", "KEKE", "OKADA"]
categories ← ["Overcharging", "Short-routing", "Reckless driving", "Harassment", "Overloading"]

// ================ MAIN PROGRAM (Flowchart 1) ================
MODULE Main()
    DECLARE phone : STRING
    DECLARE choice, attempts : INTEGER
    DECLARE isValid, isVerified, running : BOOLEAN
    attempts ← 0
    isValid ← FALSE
    DISPLAY "Welcome to SaloneMove"
    // Step 1, validation: is the number correctly formed?
    WHILE attempts < MAX_LOGIN_ATTEMPTS AND isValid = FALSE DO
        DISPLAY "Enter your phone number (e.g. 076123456):"
        INPUT phone
        IF IsValidPhone(phone) THEN
            isValid ← TRUE
        ELSE
            attempts ← attempts + 1
            DISPLAY "Invalid number. Attempts left: ", MAX_LOGIN_ATTEMPTS - attempts
        END IF
    END WHILE
    IF isValid = FALSE THEN
        DISPLAY "Too many invalid attempts. Session ended."
    ELSE
        isVerified ← VerifyPhone(phone) // Step 2, verification: one-time code by SMS
        IF isVerified = FALSE THEN
            DISPLAY "Phone number not verified. Session ended."
        ELSE
            running ← TRUE
            WHILE running = TRUE DO
                DISPLAY "1. Check a Fare / Report a Problem"
                DISPLAY "2. Officer Login"
                DISPLAY "0. Exit"
                choice ← ReadChoice(0, 2)
                CASE choice OF
                    1: CheckFare(phone)
                    2: OfficerPortal(phone)
                    0: running ← FALSE
                END CASE
            END WHILE
```

```
            DISPLAY "Thank you for using SaloneMove. Waka fine!"
        END IF
    END IF
END MODULE

FUNCTION IsValidPhone(phone : STRING) RETURNS BOOLEAN
    // Validation: checks the format only; it cannot prove who holds the phone.
    RETURN LENGTH(phone) = PHONE_LENGTH AND IsAllDigits(phone)
        AND CONTAINS(validPrefixes, SUBSTRING(phone, 0, 3))
END FUNCTION

FUNCTION VerifyPhone(phone : STRING) RETURNS BOOLEAN
    // Verification: proves the caller holds the phone by texting it a one-time code.
    // The code is a local variable, so it is never stored.
    DECLARE code, entry : STRING
    DECLARE tries, elapsed : INTEGER
    DECLARE sentTime : DATETIME
    DECLARE sent, verified : BOOLEAN
    tries ← 0
    verified ← FALSE
    code ← PAD_LEFT(RANDOM_INTEGER(0, 999999), 6, "0")
    sent ← SendSMS(phone, "SaloneMove code: " + code)
    IF sent = FALSE THEN
        DISPLAY "Code could not be sent. Please try again later."
    ELSE
        sentTime ← GetCurrentDateTime()
        WHILE tries < MAX_LOGIN_ATTEMPTS AND verified = FALSE DO
            DISPLAY "Enter the 6-digit code sent to your phone:"
            INPUT entry
            elapsed ← SECONDS_BETWEEN(sentTime, GetCurrentDateTime())
            IF entry = code AND elapsed ≤ CODE_EXPIRY_SECONDS THEN
                verified ← TRUE
            ELSE
                tries ← tries + 1
                DISPLAY "Wrong or expired code. Attempts left: ", MAX_LOGIN_ATTEMPTS - tries
            END IF
        END WHILE
    END IF
    RETURN verified
END FUNCTION

FUNCTION ReadChoice(low : INTEGER, high : INTEGER) RETURNS INTEGER
    // Shared: reads a menu choice as text and asks again until it is a whole
    // number from low to high, so letters can never reach TO_INTEGER().
    DECLARE entry : STRING
    INPUT entry
    WHILE NOT IsAllDigits(entry) OR TO_INTEGER(entry) < low
        OR TO_INTEGER(entry) > high DO
        DISPLAY "Invalid option. Enter a number from ", low, " to ", high, ":"
        INPUT entry
    END WHILE
    RETURN TO_INTEGER(entry)
END FUNCTION

// ================ CHECK A FARE (Flowchart 2) ================
```

```
MODULE CheckFare(phone : STRING)
    // The officialFare ≤ 0 test stops a damaged record causing division by zero.
    DECLARE origin, destination, route, status : STRING
    DECLARE vehicleChoice, index, answer : INTEGER
    DECLARE official, amountCharged, difference, percent : REAL
    origin ← ReadPlace("Enter starting point:")
    destination ← ReadPlace("Enter destination:")
    DISPLAY "Vehicle: 1. Poda-poda  2. Keke  3. Okada"
    vehicleChoice ← ReadChoice(1, 3)
    index ← FindFare(origin, destination, vehicleTypes[vehicleChoice - 1])
    IF index = -1 THEN
        DISPLAY "Route not found. Check the spelling or try another vehicle."
    ELSE IF fareTable[index].officialFare ≤ 0 THEN
        DISPLAY "Fare record error. Please try again later."
    ELSE
        official ← fareTable[index].officialFare
        route ← fareTable[index].route
        status ← "NOT CHECKED"
        DISPLAY "Official fare: NLe ", official
        DISPLAY "Enter the amount you paid (0 to skip):"
        amountCharged ← ReadNumber(0, official * MAX_AMOUNT_FACTOR)
        IF amountCharged > 0 THEN
            difference ← ROUND(amountCharged - official, 2)
            percent ← difference / official * 100
            status ← ClassifyFare(difference, percent)
            IF status = "CORRECT" THEN
                DISPLAY "Correct fare."
            ELSE IF status = "UNDERPAID" THEN
                DISPLAY "UNDERPAID by NLe ", ABS(difference), ". Please pay the official fare."
            ELSE
                DISPLAY status, " by NLe ", difference, " (", ROUND(percent, 0), "%)"
            END IF
        END IF
        DISPLAY "Report a problem with this vehicle?"
        DISPLAY "1. Yes  2. No"
        answer ← ReadChoice(1, 2)
        IF answer = 1 THEN
            ReportProblem(phone, route, amountCharged, status)
        END IF
    END IF
END MODULE

FUNCTION FindFare(origin : STRING, destination : STRING, vehicleType : STRING) RETURNS INTEGER
    // Routes are compared in capitals, so "lumley" matches "Lumley".
    DECLARE forward, reverse : STRING
    DECLARE i, index : INTEGER
    DECLARE sameVehicle, sameRoute : BOOLEAN
    forward ← UPPERCASE(origin + " - " + destination)
    reverse ← UPPERCASE(destination + " - " + origin)
    index ← -1
    i ← 0
    WHILE i < LENGTH(fareTable) AND index = -1 DO
        sameVehicle ← fareTable[i].vehicleType = vehicleType
        sameRoute ← UPPERCASE(fareTable[i].route) = forward
            OR UPPERCASE(fareTable[i].route) = reverse
```

```
        IF sameVehicle AND sameRoute THEN
            index ← i
        END IF
        i ← i + 1
    END WHILE
    RETURN index
END FUNCTION

FUNCTION ClassifyFare(difference : REAL, percent : REAL) RETURNS STRING
    // difference = amount paid − official fare. More than NLe 0.50 below is UNDERPAID,
    // within NLe 0.50 either way is CORRECT, and more than NLe 0.50 above is an overcharge.
    DECLARE status : STRING
    IF difference < -FARE_TOLERANCE THEN
        status ← "UNDERPAID"
    ELSE IF difference ≤ FARE_TOLERANCE THEN
        status ← "CORRECT"
    ELSE IF percent ≥ SERIOUS_PERCENT THEN
        status ← "SERIOUS OVERCHARGE"
    ELSE
        status ← "OVERCHARGED"
    END IF
    RETURN status
END FUNCTION

FUNCTION ReadPlace(prompt : STRING) RETURNS STRING
    // Shared: asks again until the place name has at least 2 letters and no digits.
    DECLARE entry : STRING
    DISPLAY prompt
    INPUT entry
    WHILE LENGTH(TRIM(entry)) < MIN_PLACE_LENGTH
        OR NOT IsLettersAndSpaces(entry) DO
        DISPLAY "Invalid place name. Use letters only, e.g. Lumley:"
        INPUT entry
    END WHILE
    RETURN TRIM(entry)
END FUNCTION

FUNCTION ReadNumber(low : REAL, high : REAL) RETURNS REAL
    // Shared: asks again until the entry is a number from low to high, so text such
    // as "ten" and typing slips such as 610 for 6.10 are caught.
    DECLARE entry : STRING
    INPUT entry
    WHILE NOT IsNumber(entry) OR TO_REAL(entry) < low
        OR TO_REAL(entry) > high DO
        DISPLAY "Invalid number. Enter a value from ", low, " to ", high, ":"
        INPUT entry
    END WHILE
    RETURN ROUND(TO_REAL(entry), 2)
END FUNCTION

// ============== REPORT A PROBLEM (Flowchart 3) ==============
MODULE ReportProblem(phone : STRING, route : STRING, amountCharged : REAL, fareStatus :
    STRING)
    // Overcharging is accepted only when the fare check found an overcharge.
    DECLARE categoryChoice : INTEGER
```

```
    DECLARE plateNumber, refID : STRING
    DECLARE overcharged, sent : BOOLEAN
    DECLARE newComplaint : Complaint
    overcharged ← fareStatus = "OVERCHARGED"
        OR fareStatus = "SERIOUS OVERCHARGE"
    DISPLAY "Problem type: 1. Overcharging"
    DISPLAY "2. Short-routing  3. Reckless driving"
    DISPLAY "4. Harassment  5. Overloading"
    categoryChoice ← ReadChoice(1, 5)
    IF categoryChoice = 1 AND overcharged = FALSE THEN
        DISPLAY "Report refused: the fare check did not find an overcharge."
    ELSE
        DISPLAY "Enter the vehicle plate number (0 if not seen):"
        INPUT plateNumber
        WHILE plateNumber ≠ "0" AND NOT IsValidPlate(plateNumber) DO
            DISPLAY "Invalid plate number. Enter it again (0 if not seen):"
            INPUT plateNumber
        END WHILE
        reportCounter ← reportCounter + 1
        refID ← "SLM-" + YEAR(GetCurrentDateTime()) + "-"
            + PAD_LEFT(reportCounter, 6, "0")
        newComplaint.referenceID ← refID
        newComplaint.phoneNumber ← phone
        newComplaint.plateNumber ← REMOVE_SPACES(UPPERCASE(plateNumber))
        newComplaint.route ← route
        newComplaint.category ← categories[categoryChoice - 1]
        newComplaint.amountCharged ← amountCharged
        newComplaint.fareStatus ← fareStatus
        newComplaint.dateTime ← GetCurrentDateTime()
        newComplaint.status ← "OPEN"
        ADD newComplaint TO complaints
        sent ← SendSMS(phone, "SaloneMove: Report received. Ref: " + refID)
        IF sent = FALSE THEN
            DISPLAY "SMS not sent. Please write down your reference."
        END IF
        DISPLAY "Thank you. Your reference number is ", refID
    END IF
END MODULE

FUNCTION IsValidPlate(plate : STRING) RETURNS BOOLEAN
    DECLARE cleaned : STRING
    cleaned ← REMOVE_SPACES(UPPERCASE(plate))
    RETURN LENGTH(cleaned) ≥ 5 AND LENGTH(cleaned) ≤ 8
        AND IsLettersAndDigits(cleaned)
END FUNCTION

// ============== OFFICER PORTAL (Flowchart 4) ==============
MODULE OfficerPortal(phone : STRING)
    // Three wrong PINs lock the portal for that phone number, so the PIN
    // cannot be found by guessing.
    DECLARE pin : STRING
    DECLARE officerChoice, tries : INTEGER
    DECLARE inPortal : BOOLEAN
    tries ← 0
    inPortal ← FALSE
```

```
    IF CONTAINS(lockedNumbers, phone) THEN
        DISPLAY "Officer login is locked for this number."
    ELSE
        WHILE tries < MAX_LOGIN_ATTEMPTS AND inPortal = FALSE DO
            DISPLAY "Enter officer PIN:"
            INPUT pin
            IF HASH(pin) = officerPINHash THEN
                inPortal ← TRUE
            ELSE
                tries ← tries + 1
                DISPLAY "Wrong PIN. Attempts left: ", MAX_LOGIN_ATTEMPTS - tries
            END IF
        END WHILE
        IF inPortal = FALSE THEN
            ADD phone TO lockedNumbers
            DISPLAY "Access denied. Officer login locked for this number."
        END IF
        WHILE inPortal = TRUE DO
            DISPLAY "1. Hotspot Report"
            DISPLAY "2. Apply Fare Change"
            DISPLAY "0. Back"
            officerChoice ← ReadChoice(0, 2)
            IF officerChoice = 1 THEN
                GenerateHotspotReport()
            ELSE IF officerChoice = 2 THEN
                ApplyFareAdjustment()
            ELSE
                inPortal ← FALSE
            END IF
        END WHILE
    END IF
END MODULE

MODULE GenerateHotspotReport()
    DECLARE r, c, count, hotspotCount : INTEGER
    DECLARE openOnRoute : BOOLEAN
    hotspotCount ← 0
    FOR r ← 0 TO LENGTH(routeList) - 1 DO
        count ← 0
        FOR c ← 0 TO LENGTH(complaints) - 1 DO
            openOnRoute ← complaints[c].route = routeList[r]
                AND complaints[c].status = "OPEN"
            IF openOnRoute THEN
                count ← count + 1
            END IF
        END FOR
        IF count ≥ HOTSPOT_THRESHOLD THEN
            DISPLAY routeList[r], ": ", count, " open complaints - HOTSPOT"
            hotspotCount ← hotspotCount + 1
        END IF
    END FOR
    IF hotspotCount = 0 THEN
        DISPLAY "No hotspots this period."
    ELSE
        DISPLAY "Total hotspots: ", hotspotCount
    END IF
```

```
END MODULE

MODULE ApplyFareAdjustment()
    // ReadNumber() keeps the change within ±50%, so no fare can fall to 0.
    DECLARE percentChange : REAL
    DECLARE confirm, i : INTEGER
    DISPLAY "Fare change in % (e.g. 9.4 or -12.5):"
    percentChange ← ReadNumber(-MAX_FARE_CHANGE, MAX_FARE_CHANGE)
    IF percentChange = 0 THEN
        DISPLAY "Rejected: the change cannot be 0%."
    ELSE
        DISPLAY "Apply ", percentChange, "% to all fares?"
        DISPLAY "1. Yes  2. No"
        confirm ← ReadChoice(1, 2)
        IF confirm = 1 THEN
            FOR i ← 0 TO LENGTH(fareTable) - 1 DO
                fareTable[i].officialFare ← ROUND(fareTable[i].officialFare
                    * (1 + percentChange / 100), 2)
            END FOR
            DISPLAY LENGTH(fareTable), " fares updated."
        ELSE
            DISPLAY "No changes made."
        END IF
    END IF
END MODULE

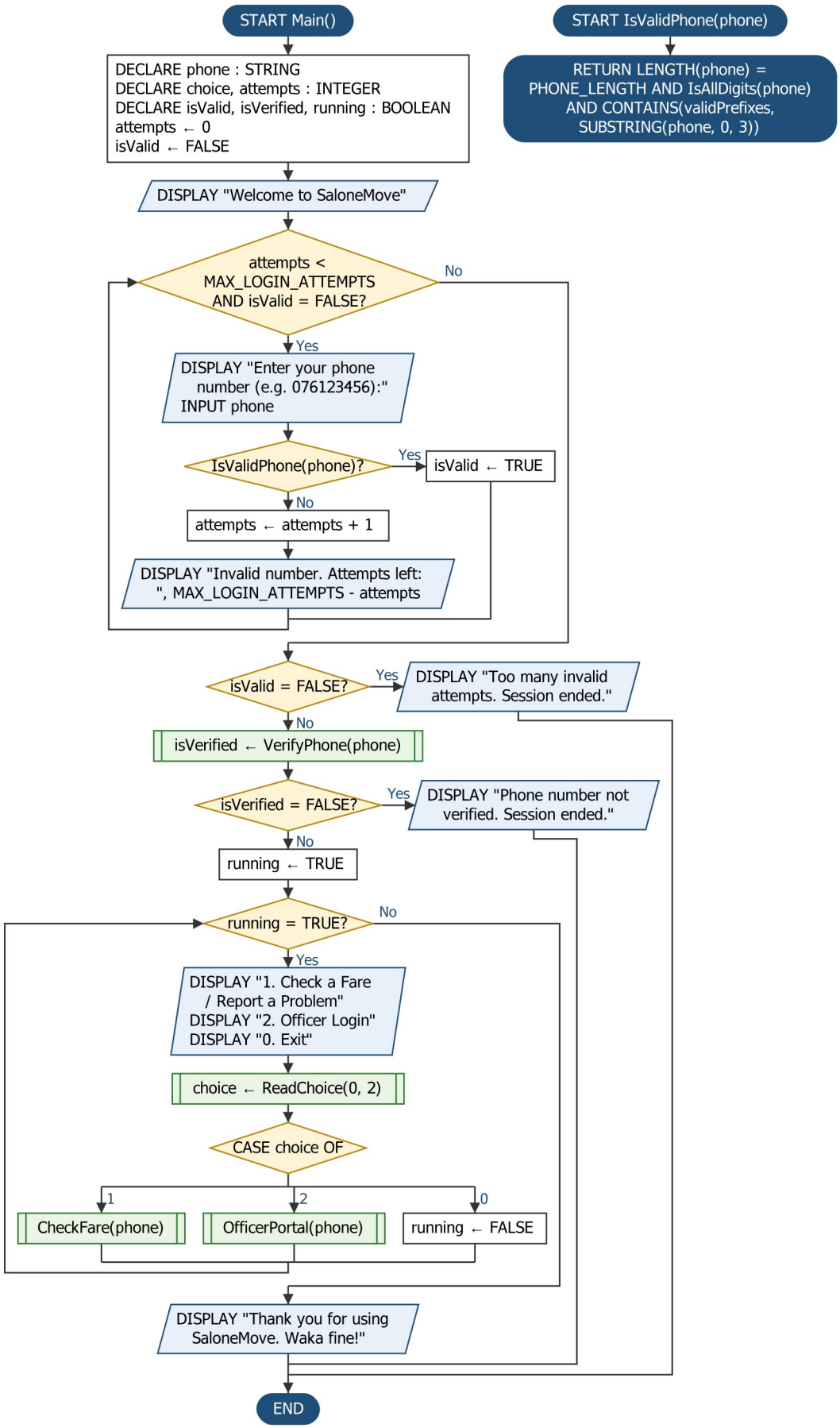
END PROGRAM
```

4. Flowcharts

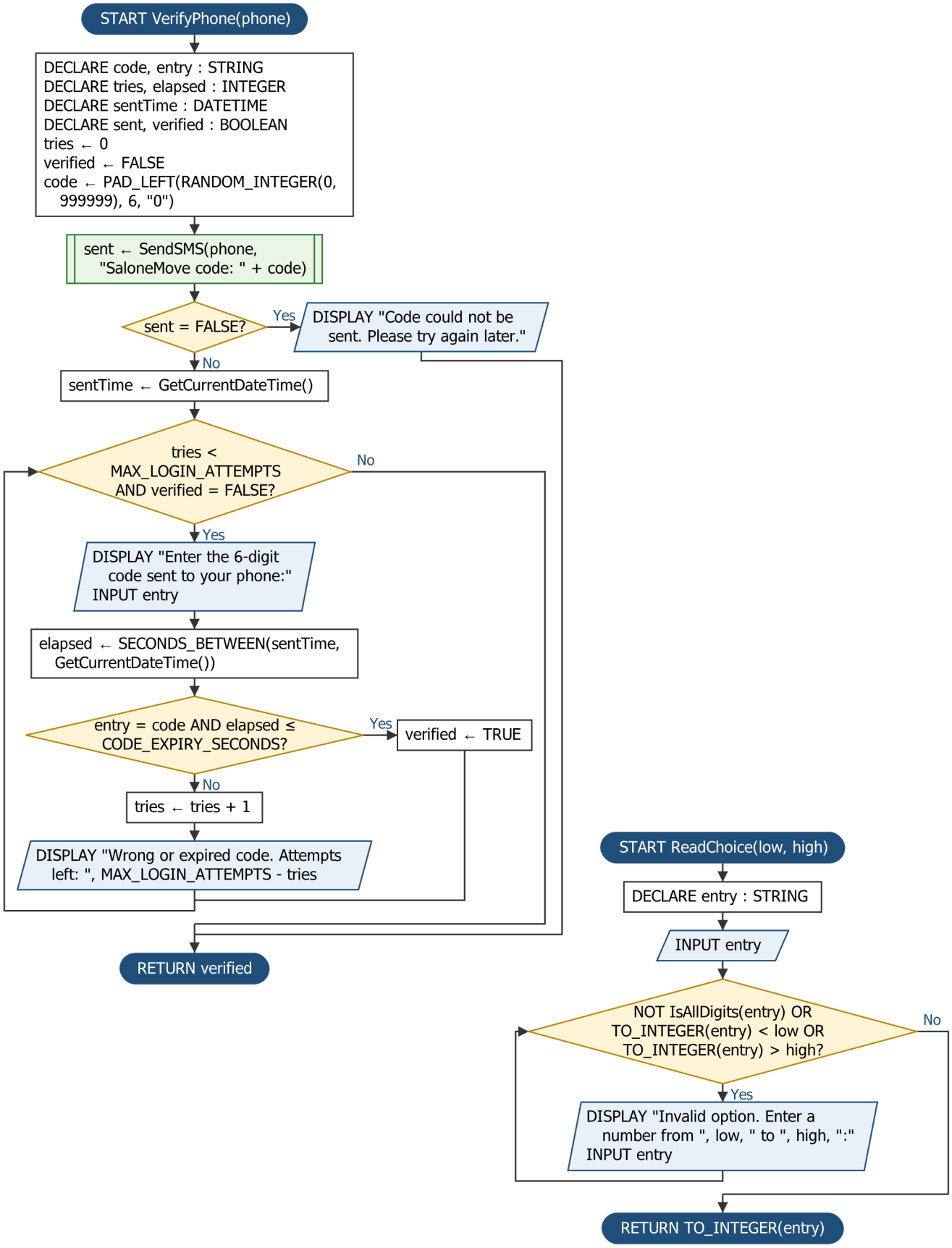
There are four flowcharts, one for each part of the system: the main program, checking a fare, reporting a problem and the officer portal. Each flowchart shows a module together with the functions or sub-modules it calls, drawn as separate START … RETURN flows on the same page. A shared input function is drawn once, in the flowchart where it is first used: ReadChoice() in Flowchart 1, and ReadPlace() and ReadNumber() in Flowchart 2. Flowcharts 1, 2 and 4 hold more procedures than fit on one page, so each continues on a second page. Every box uses exactly the same statements, names and messages as the pseudocode, so the two can be read side by side.

The flowcharts were drawn in diagrams.net (draw.io). The .drawio files are in the GitHub repository and can be opened and edited at app.diagrams.net. Each flowchart was desk-checked by following it box by box with the test data in Section 6; together, the 12 test scenarios and 4 error-handling tests pass through every box and every Yes/No and CASE path at least once.

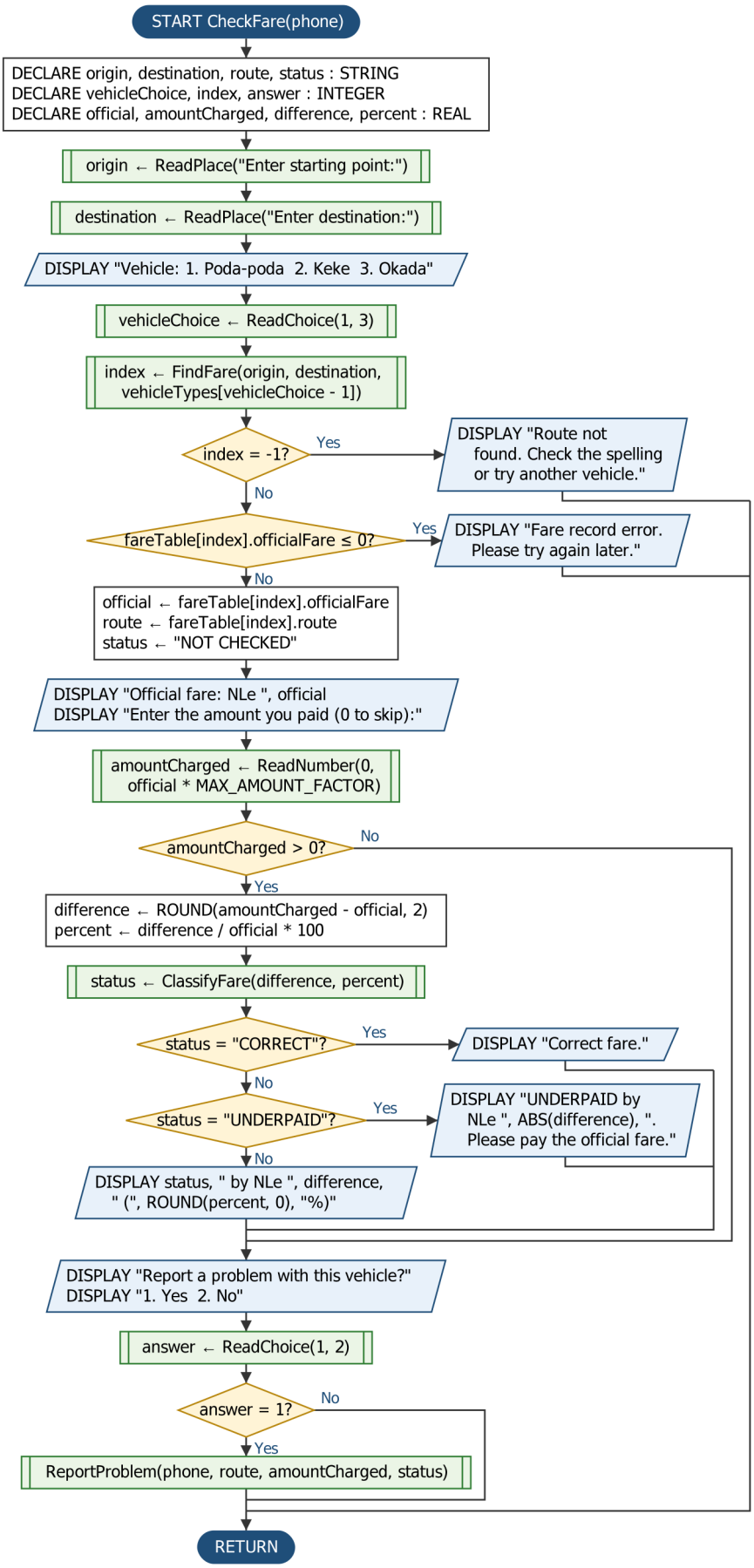
Symbol	Meaning
Rounded box (dark blue)	Terminal: START, END or RETURN
Parallelogram (light blue)	Input or output: DISPLAY, INPUT
Rectangle (white)	Process: DECLARE, assignment (←), ADD
Diamond (yellow)	Decision, loop test or CASE selection; the Yes/No or CASE value is written on each arrow
Double-sided box (green)	Predefined process: a call to another module or function
Arrow	Order of steps; an arrow entering the left point of a diamond returns to the top of a loop



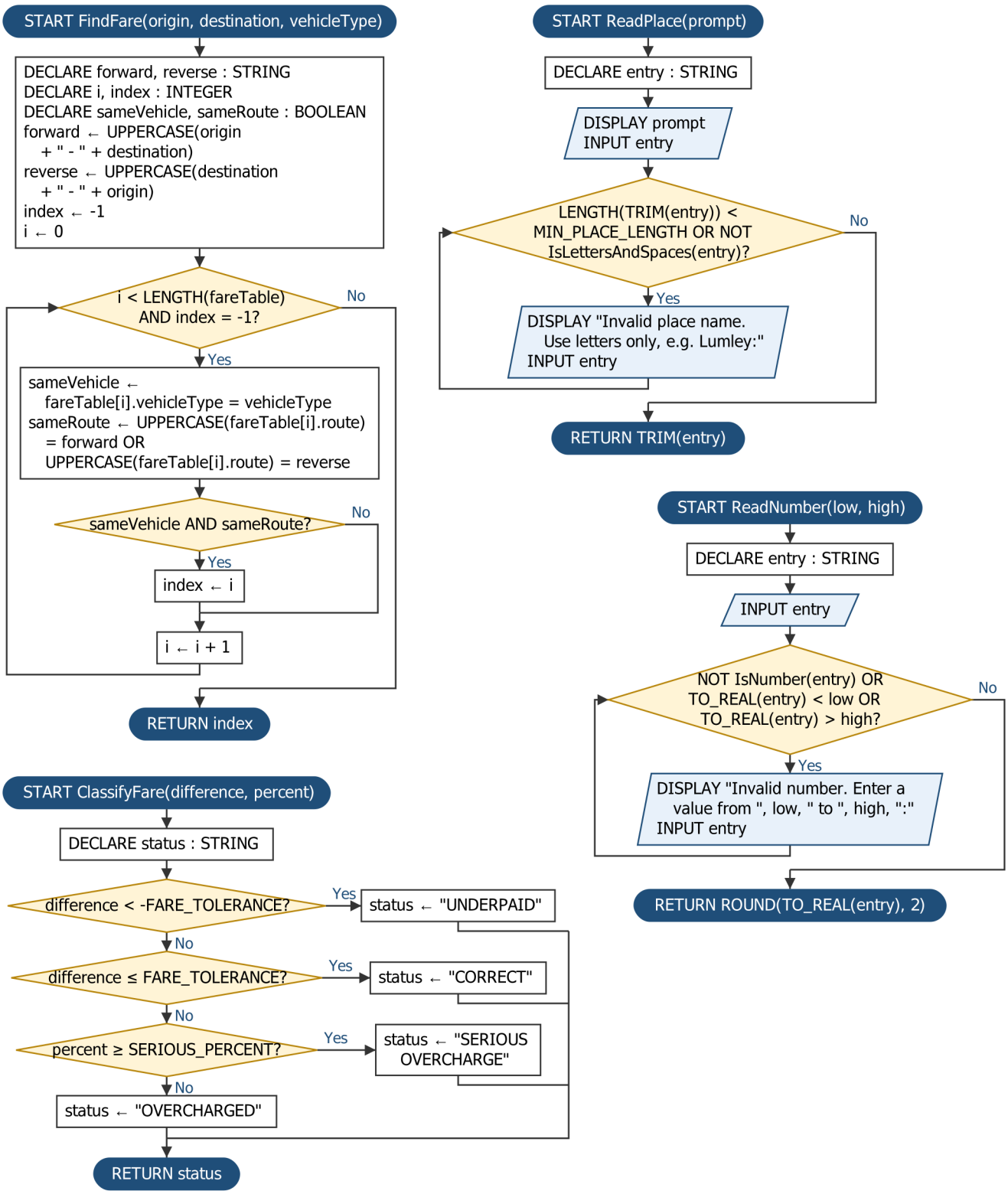
Flowchart 1 (part 1 of 2): Main program, Main() and IsValidPhone()



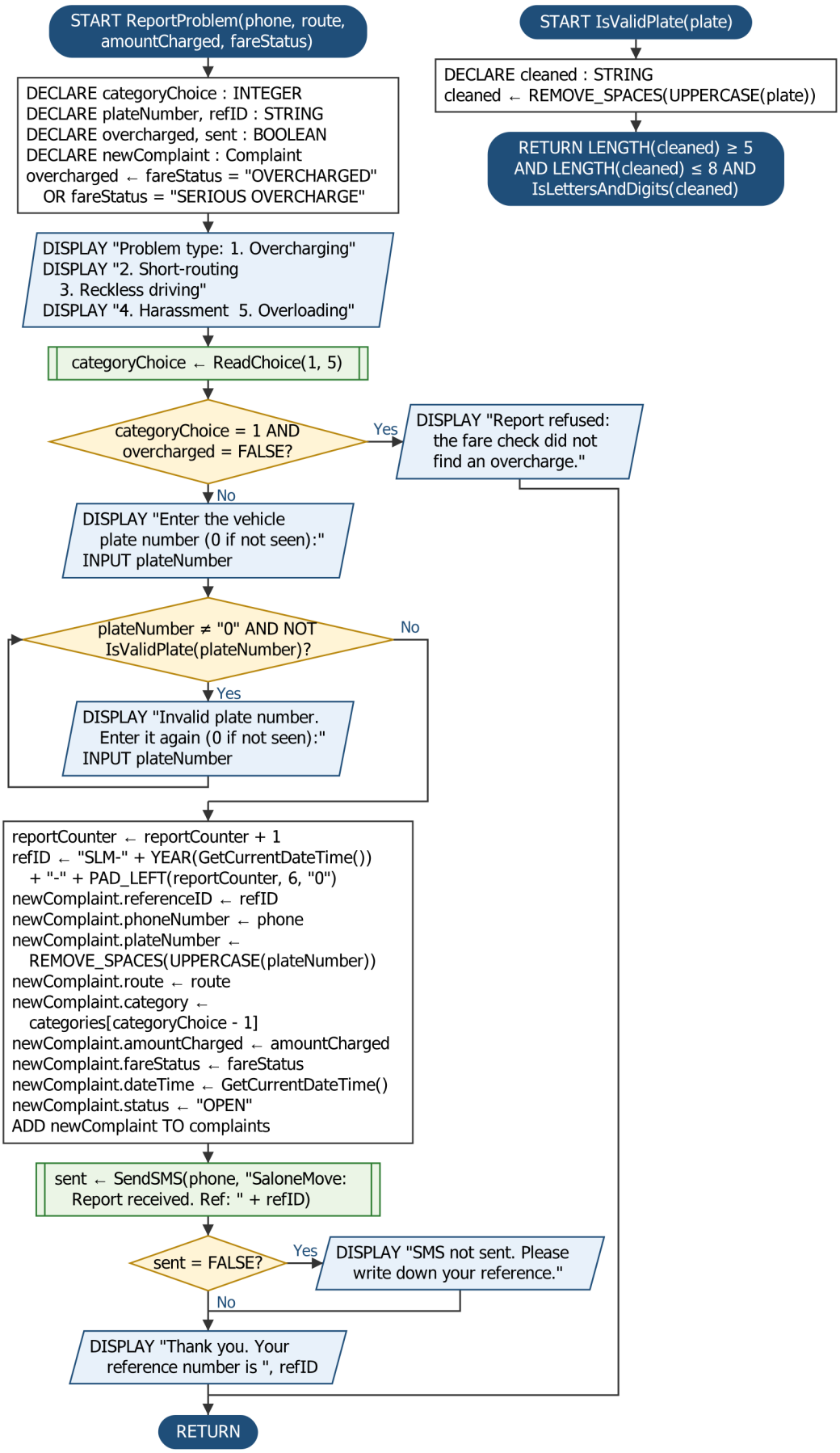
Flowchart 1 (part 2 of 2): Main program, VerifyPhone() and ReadChoice()



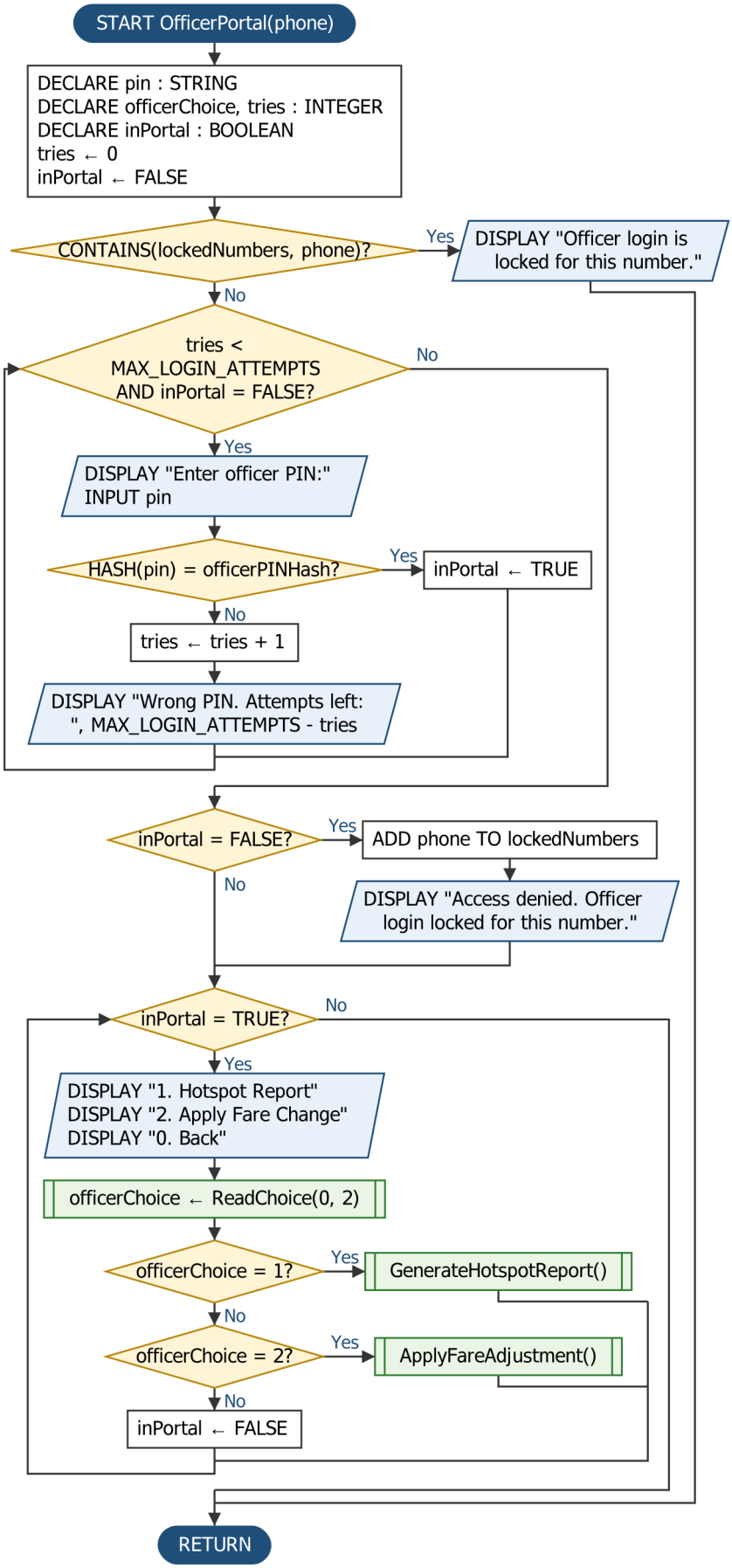
Flowchart 2 (part 1 of 2): Check a Fare, CheckFare()



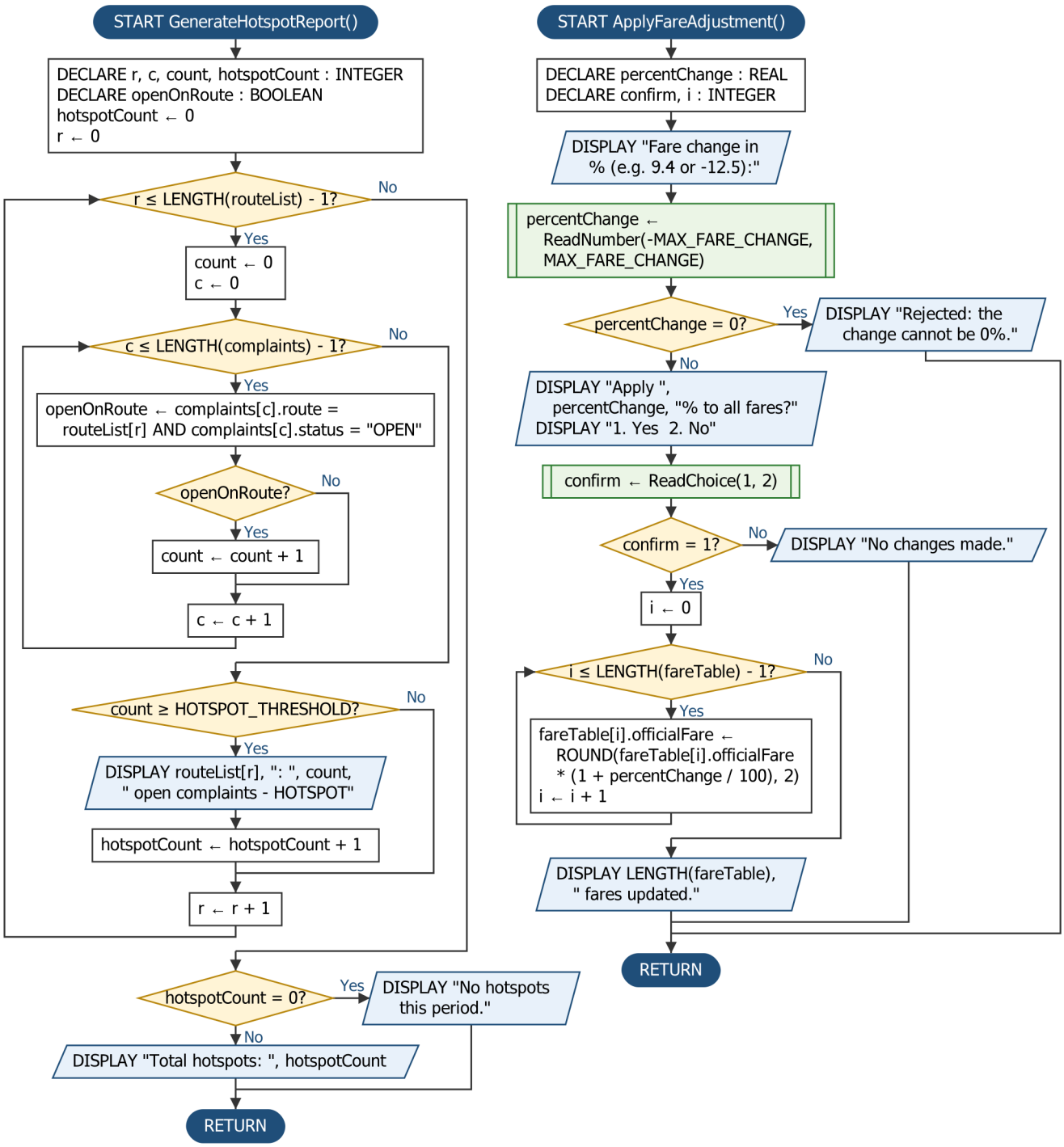
Flowchart 2 (part 2 of 2): Check a Fare, FindFare(), ClassifyFare(), ReadPlace() and ReadNumber()



Flowchart 3: Report a Problem, ReportProblem() and IsValidPlate()



Flowchart 4 (part 1 of 2): Officer Portal, OfficerPortal()



Flowchart 4 (part 2 of 2): Officer Portal, GenerateHotspotReport() and ApplyFareAdjustment()

5. Functional Modules

SaloneMove has six modules and eight functions. A module carries out a task; a function returns one value to the module that called it. Figure 1 shows which part calls which.

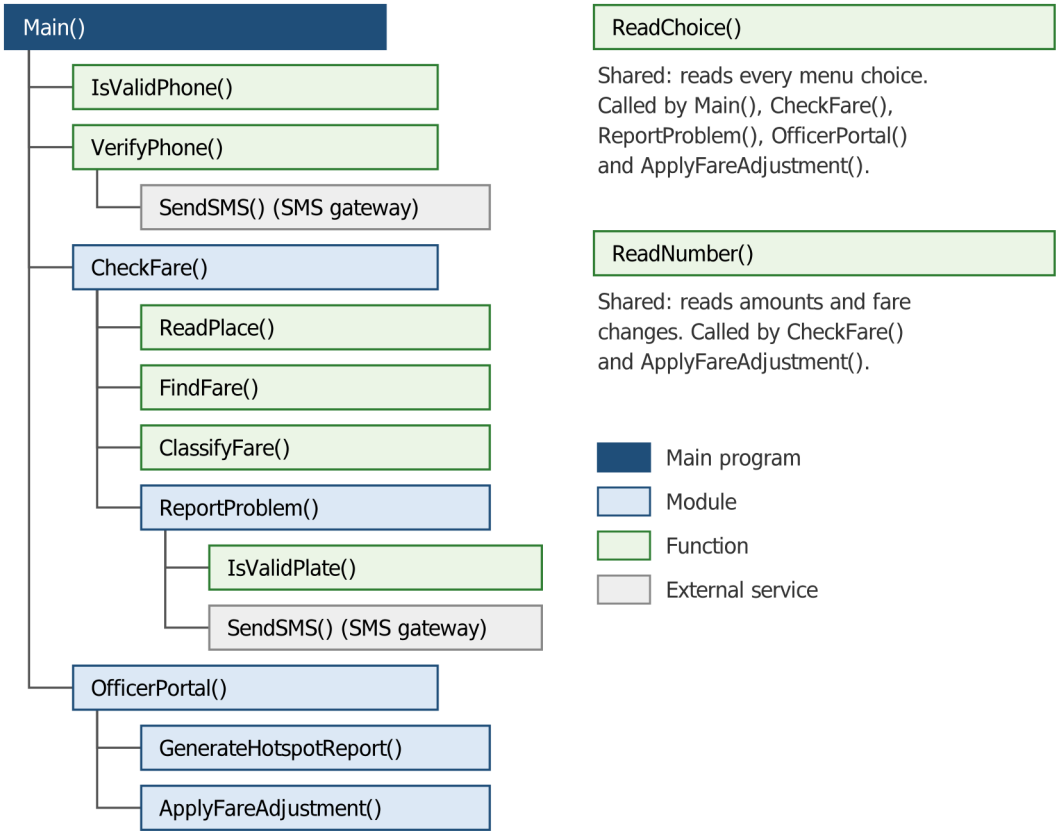


Figure 1: Structure chart (dark blue: main program; light blue: modules; green: functions; grey: external service)

5.1 Modules

Module	Input	Processing	Output
Main()	Phone number, verification code, menu choice	Allows 3 attempts at a valid number, then 3 tries at the SMS code; loops through the menu using CASE until the user chooses 0	Menu screens, goodbye or session-ended message
CheckFare()	Places, vehicle choice, amount paid, report answer	Finds the fare record and rejects a missing or damaged one; calculates difference and percentage; classifies the fare	Official fare, fare status, optional report
ReportProblem()	Problem type, plate number	Refuses an overcharging report the fare check does not support; checks the plate; numbers the report; saves a Complaint record	Saved complaint, SMS and on-screen reference

Module	Input	Processing	Output
OfficerPortal()	Officer PIN, officer menu choice	Checks the PIN hash with 3 attempts, then locks the number; loops through the officer menu	Access to the two officer modules
GenerateHotspotReport()	Route list, complaints list	Nested loops count OPEN complaints for each route and compare with the threshold	Hotspot routes and total
ApplyFareAdjustment()	Percentage change, confirmation	Rejects a 0% change; updates every fare in a loop	Updated fare table and count

5.2 Functions

Function	Parameters	Returns	Purpose
IsValidPhone()	phone : STRING	BOOLEAN	Validation: TRUE if the number has 9 digits and starts with a valid mobile prefix
VerifyPhone()	phone : STRING	BOOLEAN	Verification: TRUE if the code texted to the number is typed correctly within 3 tries and 5 minutes
ReadChoice()	low, high : INTEGER	INTEGER	A menu choice from low to high
FindFare()	origin, destination, vehicleType : STRING	INTEGER	Position of the matching fare record, or −1 if none
ClassifyFare()	difference, percent : REAL	STRING	"UNDERPAID", "CORRECT", "OVERCHARGED" or "SERIOUS OVERCHARGE"
ReadPlace()	prompt : STRING	STRING	A place name of at least 2 letters, with no digits
ReadNumber()	low, high : REAL	REAL	A number from low to high, rounded to 2 decimal places
IsValidPlate()	plate : STRING	BOOLEAN	TRUE if 5 to 8 letters and digits once spaces are removed

SendSMS() is provided by the mobile network's SMS gateway rather than by SaloneMove itself, and returns TRUE when the gateway accepts the message. The other built-in routines (LENGTH, SUBSTRING, UPPERCASE, TRIM, REMOVE_SPACES, IsAllDigits, IsNumber, IsLettersAndDigits, IsLettersAndSpaces, TO_INTEGER, TO_REAL, CONTAINS, ROUND, ABS, PAD_LEFT, RANDOM_INTEGER, YEAR, SECONDS_BETWEEN, HASH and GetCurrentDateTime) are listed at the top of the pseudocode.

5.3 Variables, Constants and Data Types

Constants and global data

Name	Kind	Data Type	Purpose
MAX_LOGIN_ATTEMPTS	Constant	INTEGER	3 attempts for the phone number, the SMS code and the officer PIN
PHONE_LENGTH	Constant	INTEGER	9 digits, e.g. 076123456
CODE_EXPIRY_SECONDS	Constant	INTEGER	300 seconds (5 minutes) before a one-time code expires
MIN_PLACE_LENGTH	Constant	INTEGER	2, the shortest place name accepted
FARE_TOLERANCE	Constant	REAL	NLe 0.50 allowed either way for lack of small change
SERIOUS_PERCENT	Constant	REAL	50%, the point where an overcharge is serious
MAX_AMOUNT_FACTOR	Constant	REAL	10: the amount paid may be at most 10 times the official fare
HOTSPOT_THRESHOLD	Constant	INTEGER	3 open complaints make a route a hotspot
MAX_FARE_CHANGE	Constant	REAL	50%, the largest fare change accepted at once
FareRecord, Complaint	Record types	RECORD	One fare (route, vehicle type, fare) and one complaint (now also storing the fare status)
fareTable	Global data	ARRAY OF FareRecord	The official fare list
routeList	Global data	ARRAY OF STRING	Every route in the fare table, listed once
complaints	Global data	ARRAY OF Complaint	All reports with reference, route, problem type and status
validPrefixes	Global data	ARRAY OF STRING	Mobile prefixes in the national numbering plan
lockedNumbers	Global data	ARRAY OF STRING	Numbers locked out of the officer portal after 3 wrong PINs
vehicleTypes, categories	Global data	ARRAY OF STRING	The 3 vehicle codes and the 5 problem types
officerPINHash	Global data	STRING	The officer PIN, stored only as a hash
reportCounter	Global data	INTEGER	Number of reports issued so far

Local variables and parameters

Module or function	Parameters	Local variables and data types
Main()	none	phone : STRING; choice, attempts : INTEGER; isValid, isVerified, running : BOOLEAN
IsValidPhone()	phone : STRING	none

Module or function	Parameters	Local variables and data types
VerifyPhone()	phone : STRING	code, entry : STRING; tries, elapsed : INTEGER; sentTime : DATETIME; sent, verified : BOOLEAN
ReadChoice()	low : INTEGER; high : INTEGER	entry : STRING
CheckFare()	phone : STRING	origin, destination, route, status : STRING; vehicleChoice, index, answer : INTEGER; official, amountCharged, difference, percent : REAL
FindFare()	origin : STRING; destination : STRING; vehicleType : STRING	forward, reverse : STRING; i, index : INTEGER; sameVehicle, sameRoute : BOOLEAN
ClassifyFare()	difference : REAL; percent : REAL	status : STRING
ReadPlace()	prompt : STRING	entry : STRING
ReadNumber()	low : REAL; high : REAL	entry : STRING
ReportProblem()	phone : STRING; route : STRING; amountCharged : REAL; fareStatus : STRING	categoryChoice : INTEGER; plateNumber, refID : STRING; overcharged, sent : BOOLEAN; newComplaint : Complaint
IsValidPlate()	plate : STRING	cleaned : STRING
OfficerPortal()	phone : STRING	pin : STRING; officerChoice, tries : INTEGER; inPortal : BOOLEAN
GenerateHotspotReport()	none	r, c, count, hotspotCount : INTEGER; openOnRoute : BOOLEAN
ApplyFareAdjustment()	none	percentChange : REAL; confirm, i : INTEGER

5.4 Decision Structures

Structure	Where it is used	Purpose
IF ... THEN	CheckFare(): amountCharged > 0, answer = 1; FindFare(): sameVehicle AND sameRoute; ReportProblem(): sent = FALSE; OfficerPortal(): inPortal = FALSE; GenerateHotspotReport(): openOnRoute, count ≥ HOTSPOT_THRESHOLD	Does something only when a condition is true
IF ... THEN ... ELSE	Main(): IsValidPhone(phone), isValid = FALSE, isVerified = FALSE; VerifyPhone(): sent = FALSE, code check; ReportProblem(): overcharging check; OfficerPortal(): locked number, PIN check; ApplyFareAdjustment(): percentChange = 0, confirm = 1; GenerateHotspotReport(): hotspotCount = 0	Chooses between two paths
Nested IF	Main(): verification inside the validation test; CheckFare(): status test inside amountCharged > 0, inside the record checks; OfficerPortal(): PIN loop inside the lock test; ApplyFareAdjustment(): confirm test inside the zero check	Makes a second decision only after the first

Structure	Where it is used	Purpose
IF … ELSE IF … ELSE	ClassifyFare(): four fare statuses; CheckFare(): route not found, fare record error or fare check, and the message for each status; OfficerPortal() menu	Selects one of several outcomes
CASE	Main() menu	Selects one option from a numbered menu

5.5 Repetition Structures

Structure	Where it is used	Purpose
WHILE with a counter and a flag	Main(): attempts < MAX_LOGIN_ATTEMPTS AND isValid = FALSE; VerifyPhone(): tries < MAX_LOGIN_ATTEMPTS AND verified = FALSE; OfficerPortal(): tries < MAX_LOGIN_ATTEMPTS AND inPortal = FALSE	Allows up to 3 attempts, stopping early on success
WHILE controlled by a flag	Main(): running; OfficerPortal(): inPortal	Keeps a menu running until the user leaves
WHILE search with early exit	FindFare(): i < LENGTH(fareTable) AND index = −1	Stops searching as soon as a match is found
WHILE with a priming read	ReadChoice(), ReadNumber(), ReadPlace(); ReportProblem(): plate number	Asks again only while the entry is invalid
FOR	ApplyFareAdjustment(): every fare	Visits every record a known number of times
Nested FOR	GenerateHotspotReport(): every complaint for every route	Counts matches for each route

5.6 Boolean Logic

Expression	Meaning
LENGTH(phone) = PHONE_LENGTH AND IsAllDigits(phone) AND CONTAINS(validPrefixes, SUBSTRING(phone, 0, 3))	All three must be true for a valid mobile number
attempts < MAX_LOGIN_ATTEMPTS AND isValid = FALSE	Keep asking only while attempts remain and no valid number has been entered
entry = code AND elapsed ≤ CODE_EXPIRY_SECONDS	The code must be right and typed in time
NOT IsAllDigits(entry) OR TO_INTEGER(entry) < low OR TO_INTEGER(entry) > high	Ask again unless the entry is a whole number in range; OR stops at the first TRUE, so letters never reach TO_INTEGER()
LENGTH(TRIM(entry)) < MIN_PLACE_LENGTH OR NOT IsLettersAndSpaces(entry)	Ask again if the place name is too short or holds digits or symbols

Expression	Meaning
sameRoute ← UPPERCASE(fareTable[i].route) = forward OR UPPERCASE(fareTable[i].route) = reverse	Lumley to Jui and Jui to Lumley match the same record, however the places are capitalised
sameVehicle AND sameRoute	A record matches only if both are true
difference < -FARE_TOLERANCE, then difference ≤ FARE_TOLERANCE	Together the two tests split payments into underpaid, correct and overcharged
categoryChoice = 1 AND overcharged = FALSE	Refuses an overcharging report that the fare check does not support
plateNumber ≠ "0" AND NOT IsValidPlate(plateNumber)	Ask again only if a plate was entered and it is invalid
openOnRoute ← complaints[c].route = routeList[r] AND complaints[c].status = "OPEN"	Closed complaints are not counted
HASH(pin) = officerPINHash	Grants access only when the PIN is right

5.7 Input Validation and Error Handling

Every value a user types is checked before it is used. Ordinary entries are asked again with a message until they are valid; the phone number, the SMS code and the officer PIN allow only 3 attempts, because repeated guessing there is a security risk. The first table lists each entry and its rule; the second lists the other errors the design handles.

Entry	Checked by	Rule	If it fails
Phone number	Main(), IsValidPhone()	9 digits starting with a valid mobile prefix	"Invalid number. Attempts left: n"; after 3, the session ends
Verification code	VerifyPhone()	Must match the code sent by SMS and be typed within 5 minutes	"Wrong or expired code..."; after 3, the session ends
Menu and yes/no answers	ReadChoice()	Digits only, within the menu's range	"Invalid option. Enter a number from ... to ..." and asked again
Starting point, destination	ReadPlace()	At least 2 letters; letters and spaces only	"Invalid place name..." and asked again
Amount paid	ReadNumber()	A number from 0 to 10 × the official fare	"Invalid number. Enter a value from ..." and asked again
Problem type	ReadChoice(), ReportProblem()	1 to 5; Overcharging only if the fare check found an overcharge	Asked again, or the report is refused
Plate number	IsValidPlate()	5 to 8 letters and digits once spaces are removed, or 0	"Invalid plate number..." and asked again
Officer PIN	OfficerPortal()	Its hash must match the stored hash	"Wrong PIN..."; after 3, the number is locked out of the portal
Fare change	ReadNumber(), ApplyFareAdjustment()	A number from −50 to 50, not 0	Asked again, or rejected with no fare changed

Situation	How it is handled
Text typed where a number is expected	IsAllDigits() or IsNumber() is tested first, and OR stops at the first TRUE, so TO_INTEGER() and TO_REAL() never receive letters
Places typed in different capitals	FindFare() compares routes in capitals, so "lumley" matches "Lumley"
Damaged fare record (official fare 0 or less)	CheckFare() stops before the percentage is calculated, so it is never divided by zero; fare changes are limited to ±50% so no fare can fall to 0
SMS gateway cannot send the code	VerifyPhone() displays a message and returns FALSE, and the session ends safely
SMS gateway cannot send a report reference	The report is still saved and the reference is shown on screen with a request to write it down
Repeated PIN guessing	After 3 wrong PINs the number is added to lockedNumbers and cannot try again

6. Expected Outputs

The scenarios below were traced through both the pseudocode and the flowcharts, and both give the outputs shown. The poda-poda fares come from the official fare list that took effect on 30 June 2026 (Sierraloaded, 2026b). That list sets one "pole to pole" fare of NLe 6.10 for kekes and okadas, so the keke and okada routes are sample entries using that fare. The complaints are invented test data, the test officer PIN is 2468 (stored only as a hash), and the date is taken as 20 October 2026. For testing, the random one-time code is fixed at 482913 and the mobile prefixes are 030, 031, 033, 034, 072 to 080, 088 and 099; the live system would load the current list from the national numbering plan.

6.1 Sample Data

Sample fare table

Index	Route	Vehicle	Official fare (NLe)
0	Lumley - Regent Road	Poda-poda	6.10
1	Regent - Bus Station	Poda-poda	8.60
2	Wilberforce - Jui	Poda-poda	13.50
3	Waterloo - Bombay Street	Poda-poda	14.70
4	Jui - Lumley	Poda-poda	19.70
5	Waterloo - Tokeh	Poda-poda	30.70
6	Lumley - Wilberforce	Keke	6.10
7	Congo Cross - Wilberforce	Keke	6.10
8	Aberdeen - Lumley	Okada	6.10
9	Model - Fourah Bay College	Okada	6.10

Sample complaints (reportCounter = 10)

Route	OPEN complaints	CLOSED complaints
Lumley - Wilberforce	4	0
Wilberforce - Jui	3	1
Waterloo - Bombay Street	2	0

6.2 Test Scenarios

Inputs are listed in the order they are typed. Scenario 1 shows the complete session; the others show only the key output lines, leaving out the menus and prompts. Lines in square brackets are text messages sent by SMS.

Scenario 1: Complete session, with places typed in reverse order and in small letters

The user enters the number, types the code from the SMS, chooses 1 and travels from Regent Road to Lumley by poda-poda, typing both places in small letters. The fare table stores this route as "Lumley - Regent Road", so FindFare() finds it through the reverse match in capitals. The user skips the amount and does not report. Lines starting with > are what the user types. **Inputs:** 076123456, 482913, 1, regent road, lumley, 1, 0, 2, 0

Complete session

Welcome to SaloneMove
Enter your phone number (e.g. 076123456):
> 076123456
[SMS to 076123456] SaloneMove code: 482913
Enter the 6-digit code sent to your phone:
> 482913
1. Check a Fare / Report a Problem
2. Officer Login
0. Exit
> 1
Enter starting point:
> regent road
Enter destination:
> lumley
Vehicle: 1. Poda-poda 2. Keke 3. Okada
> 1
Official fare: NLe 6.10
Enter the amount you paid (0 to skip):
> 0
Report a problem with this vehicle?
1. Yes 2. No
> 2
1. Check a Fare / Report a Problem
2. Officer Login
0. Exit
> 0
Thank you for using SaloneMove. Waka fine!

Scenario 2: Correct fare

Lumley to Wilberforce by keke, paid NLe 6.50. The difference is +NLe 0.40, which is within FARE_TOLERANCE, so ClassifyFare() returns "CORRECT". The trace table below follows FindFare() for this search. **Inputs:** 076123456, 482913, 1, Lumley, Wilberforce, 2, 6.50, 2, 0

Expected output

Official fare: NLe 6.10
Correct fare.

Trace table: FindFare("Lumley", "Wilberforce", "KEKE") in Scenario 2

forward = "LUMLEY - WILBERFORCE" and reverse = "WILBERFORCE - LUMLEY". The loop checks one record per pass:

i	fareTable[i].route	vehicleType	sameVehicle	sameRoute	index after the pass
0	Lumley - Regent Road	PODA	FALSE	FALSE	-1
1	Regent - Bus Station	PODA	FALSE	FALSE	-1
2	Wilberforce - Jui	PODA	FALSE	FALSE	-1
3	Waterloo - Bombay Street	PODA	FALSE	FALSE	-1
4	Jui - Lumley	PODA	FALSE	FALSE	-1
5	Waterloo - Tokeh	PODA	FALSE	FALSE	-1
6	Lumley - Wilberforce	KEKE	TRUE	TRUE	6
7	Loop ends: index ≠ -1				RETURN 6

Scenario 3: Underpaid, and an overcharging report is refused

Wilberforce to Jui by poda-poda, paid NLe 12.00 against NLe 13.50. The difference is −NLe 1.50, below −FARE_TOLERANCE, so the status is "UNDERPAID" (the first design wrongly called this a fair fare). The user then tries to report overcharging, which ReportProblem() refuses because the fare check found no overcharge. **Inputs:** 076123456, 482913, 1, Wilberforce, Jui, 1, 12.00, 1, 1, 0

Expected output

Official fare: NLe 13.50
UNDERPAID by NLe 1.50. Please pay the official fare.
Report refused: the fare check did not find an overcharge.

Scenario 4: Overcharged and reported without a plate number

Wilberforce to Jui by poda-poda, paid NLe 16.20 against NLe 13.50. The difference is NLe 2.70, which is 20%, below SERIOUS_PERCENT, so the status is "OVERCHARGED". The user reports it as overcharging and enters 0 because the plate was not seen. reportCounter goes from 10 to 11. **Inputs:** 076123456, 482913, 1, Wilberforce, Jui, 1, 16.20, 1, 1, 0, 0

Expected output

Official fare: NLe 13.50
OVERCHARGED by NLe 2.70 (20%)
[SMS to 076123456] SaloneMove: Report received. Ref: SLM-2026-000011
Thank you. Your reference number is SLM-2026-000011

Scenario 5: Serious overcharge, with wrong entries corrected

Vehicle 5 is rejected by ReadChoice(). The amounts -3 and "ten" are rejected by ReadNumber(), and 10 is accepted. NLe 10.00 against NLe 6.10 is NLe 3.90 more (63.9%), so the status is "SERIOUS OVERCHARGE". Problem type 7 is rejected, the plate AB1 is too short, and "akk 1234" is accepted and saved as AKK1234. **Inputs:** 076123456, 482913, 1, Lumley, Wilberforce, 5, 2, -3, ten, 10, 1, 7, 1, AB1, akk 1234, 0

Expected output

Invalid option. Enter a number from 1 to 3:
Official fare: NLe 6.10
Invalid number. Enter a value from 0.00 to 61.00:
Invalid number. Enter a value from 0.00 to 61.00:
SERIOUS OVERCHARGE by NLe 3.90 (64%)
Invalid option. Enter a number from 1 to 5:
Invalid plate number. Enter it again (0 if not seen):
[SMS to 076123456] SaloneMove: Report received. Ref: SLM-2026-000011
Thank you. Your reference number is SLM-2026-000011

Complaint saved in Scenario 5

Field	Value
referenceID	SLM-2026-000011
phoneNumber	076123456
plateNumber	AKK1234
route	Lumley - Wilberforce
category	Overcharging
amountCharged	10.00
fareStatus	SERIOUS OVERCHARGE
dateTime	20 Oct 2026 09:03
status	OPEN

Scenario 6: Phone number validation: two wrong numbers, then a valid one

07612 is too short and 012345678 does not start with a mobile prefix (the first design accepted it), so two attempts are used. The third number is valid, the code is sent and typed correctly, and menu choice 9 is rejected before the user exits. **Inputs:** 07612, 012345678, 076123456, 482913, 9, 0

Expected output

Invalid number. Attempts left: 2
Invalid number. Attempts left: 1
[SMS to 076123456] SaloneMove code: 482913
Invalid option. Enter a number from 0 to 2:

Scenario 7: Failed validation

The first number is too short, the second contains a letter and the third has no valid mobile prefix, so all three fail IsValidPhone() and the session ends before any code is sent. **Inputs:** 7612, 07612345A, 012345678

Expected output

Invalid number. Attempts left: 2
Invalid number. Attempts left: 1
Invalid number. Attempts left: 0
Too many invalid attempts. Session ended.

Scenario 8: Failed verification

The number is valid, so a code is sent. The first code is wrong. The second is the right code, but it is typed 6 minutes after it was sent, so elapsed is more than CODE_EXPIRY_SECONDS (300) and it is refused. The third is wrong, so VerifyPhone() returns FALSE and the session ends. This is the step that proves the caller holds the phone. **Inputs:** 076123456, 111111, 482913 (after 6 minutes), 000000

Expected output

[SMS to 076123456] SaloneMove code: 482913
Wrong or expired code. Attempts left: 2
Wrong or expired code. Attempts left: 1
Wrong or expired code. Attempts left: 0
Phone number not verified. Session ended.

Scenario 9: Invalid place name, then a route that is not listed

"123" is rejected by ReadPlace() and the user types Lumley. The keke search for Lumley to Jui finds no keke record (that route is listed for poda-podas only), so FindFare() returns −1. **Inputs:** 076123456, 482913, 1, 123, Lumley, Jui, 2, 0

Expected output

Invalid place name. Use letters only, e.g. Lumley:
Route not found. Check the spelling or try another vehicle.

Scenario 10: Officer: three wrong PINs lock the portal for that number

Three wrong PINs are entered, so OfficerPortal() adds 076123456 to lockedNumbers. When the user chooses Officer Login again, access is refused at once without asking for a PIN. **Inputs:** 076123456, 482913, 2, 1111, 1234, 9999, 2, 0

Expected output

Wrong PIN. Attempts left: 2
Wrong PIN. Attempts left: 1
Wrong PIN. Attempts left: 0
Access denied. Officer login locked for this number.
Officer login is locked for this number.

Scenario 11: Officer: hotspot report and fare change, with invalid entries

With the sample complaints, two routes have 3 or more OPEN complaints; the CLOSED complaint on Wilberforce - Jui is not counted. The officer then enters 5, which ReadChoice() rejects, and chooses 2. ReadNumber() rejects "sixty" and 60 and accepts 0, which ApplyFareAdjustment() rejects. The officer enters 9.4 but answers 2, and finally applies 9.4%. **Inputs:** 076123456, 482913, 2, 2468, 1, 5, 2, sixty, 60, 0, 2, 9.4, 2, 2, 9.4, 1, 0, 0

Expected output

Wilberforce - Jui: 3 open complaints - HOTSPOT
Lumley - Wilberforce: 4 open complaints - HOTSPOT
Total hotspots: 2
Invalid option. Enter a number from 0 to 2:
Invalid number. Enter a value from -50.00 to 50.00:
Invalid number. Enter a value from -50.00 to 50.00:
Rejected: the change cannot be 0%.
Apply 9.40% to all fares?
No changes made.
Apply 9.40% to all fares?
10 fares updated.

After the fare change every fare has been multiplied by 1.094 and rounded to 2 decimal places, for example Lumley - Regent Road 6.10 → 6.67, Wilberforce - Jui 13.50 → 14.77, Jui - Lumley 19.70 → 21.55, Waterloo - Tokeh 30.70 → 33.59.

Scenario 12: Officer: no hotspots at the start of a new period

At the start of a reporting period the complaints list is empty, so every count stays at 0 and no route reaches the threshold. **Inputs:** 076123456, 482913, 2, 2468, 1, 0, 0

Expected output

No hotspots this period.

6.3 Fare Classification Tests

ClassifyFare() was tested at and on either side of each boundary, using the keke fare of NLe 6.10 for Lumley - Wilberforce. The difference is rounded to 2 decimal places before it is classified.

Amount paid (NLe)	Difference	Percent	Status	Why this value
0	–	–	NOT CHECKED	0 means the amount was skipped, so no status is calculated
4.60	−1.50	−24.6%	UNDERPAID	Clearly below the fare
5.59	−0.51	−8.4%	UNDERPAID	Just past the lower tolerance
5.60	−0.50	−8.2%	CORRECT	Lower boundary: −0.50 is still within the tolerance
6.10	+0.00	0.0%	CORRECT	Exact fare
6.60	+0.50	8.2%	CORRECT	Upper boundary: +0.50 is still within the tolerance
6.61	+0.51	8.4%	OVERCHARGED	Just past the upper tolerance

Amount paid (NLe)	Difference	Percent	Status	Why this value
9.14	+3.04	49.8%	OVERCHARGED	Just below 50%
9.15	+3.05	50.0%	SERIOUS OVERCHARGE	Exactly 50%: the serious threshold

6.4 Error-Handling Tests

These tests cover the error paths that the scenarios do not reach. Each starts after logging in with 076123456 and code 482913, except E1, where the SMS gateway is down. E3 uses a copy of the fare table in which the Lumley - Wilberforce keke fare has been damaged to 0, and in E4 the SMS gateway fails after the code is sent.

Test	What is tested	Inputs	Key output
E1	SMS gateway down when the code is sent	076123456 (no code is sent)	Code could not be sent. Please try again later. Phone number not verified. Session ended.
E2	Amount typed as 610 instead of 6.10	1, Lumley, Wilberforce, 2, 610, 6.10, 2, 0	Official fare: NLe 6.10 Invalid number. Enter a value from 0.00 to 61.00: Correct fare.
E3	Damaged fare record (official fare 0)	1, Lumley, Wilberforce, 2, 0	Fare record error. Please try again later.
E4	SMS gateway down when a report is saved	1, Lumley, Wilberforce, 2, 0, 1, 4, 0, 0	Official fare: NLe 6.10 SMS not sent. Please write down your reference. Thank you. Your reference number is SLM-2026-000011

7. GitHub Evidence

The project is stored in a public GitHub repository named according to the required convention. Work was committed in stages as each part of the design was completed, and the changes made after the examiner's review were committed separately so they can be seen on their own.

Item	Details
Repository name	PROG101_Assignment1_[StudentName]_[StudentID]
Repository URL	https://github.com/[username]/ PROG101_Assignment1_[StudentName]_[StudentID]
Contents	README.md, LICENSE, report/, algorithm/, pseudocode/, flowcharts/ (diagrams.net files and images), screenshots/

Commit history

Commit	Message
1	Initial commit: add README and LICENSE
2	Add problem analysis and rationale
3	Add step-by-step algorithm
4	Add pseudocode for all modules and functions
5	Add the four flowcharts (diagrams.net files and images)
6	Add functional modules and expected outputs
7	Add final report (DOCX and PDF) and screenshots
8	Revise after review: phone validation and verification
9	Revise after review: underpaid, correct and overcharged fare statuses
10	Revise after review: input validation, error handling and new test scenarios

Repository structure

```
PROG101_Assignment1_[StudentName]_[StudentID]/
    README.md
    LICENSE
    report/
        SaloneMove_Report.docx
        SaloneMove_Report.pdf
    algorithm/
        SaloneMove_algorithm.md
    pseudocode/
        SaloneMove_pseudocode.txt
    flowcharts/
        flowchart1_main.drawio, _part1.png and _part2.png
        flowchart2_check_fare.drawio, _part1.png and _part2.png
        flowchart3_report_problem.drawio and .png
        flowchart4_officer_portal.drawio, _part1.png and _part2.png
        structure_chart.drawio and .png
    screenshots/
        repository_home.png
        commit_history.png
```

README.md

The README gives the project title and student details, the problem, the four services, a table of the repository folders, the programming logic covered, the SDG, open source and DPG summary, and how to open the flowcharts in diagrams.net.

[Insert repository home page showing files and README here]

Screenshot 1: Repository home page showing files and README

[Insert commit history (Commits page) here]

Screenshot 2: Commit history (Commits page)

8. References

Awoko (2013) 'Commuters grumble over poor transportation', *Awoko Newspaper*, 7 November. Available at: https://awokonewspapersl.com/sierra-leone-news-commuters-grumble-over-poor-transportation/ (Accessed: 16 October 2026).

Awoko (2015) 'Taxi drivers extort citizens with impunity', *Awoko Newspaper*, 5 May. Available at: https://awokonewspapersl.com/sierra-leone-news-taxi-drivers-extort-citizens-with-impunity/ (Accessed: 16 October 2026).

BudgIT Sierra Leone (2026) *When Fuel Prices Rise, Citizens Pay the Price*. Freetown: BudgIT. Available at: https://sierraleone.budgit.org/wp-content/uploads/2026/03/When-Fuel-Prices-Rise-Citizens-Pay-the-Price-2.pdf (Accessed: 16 October 2026).

DataReportal (2026) *Digital 2026: Sierra Leone*. Available at: https://datareportal.com/reports/digital-2026-sierra-leone (Accessed: 16 October 2026).

Digital Public Goods Alliance (n.d.) *Digital Public Goods Standard*. Available at: https://www.digitalpublicgoods.net/standard (Accessed: 16 October 2026).

Greene, G. and Morgan, S. (2024) 'Understanding Okada and Keke operations in Freetown', *Thredbo 18 Conference*. Sydney: University of Sydney. Available at: https://ses.library.usyd.edu.au/handle/2123/33382 (Accessed: 16 October 2026).

Sierraloaded (2026a) 'BREAKING: Government of Sierra Leone reduces fuel price', *Sierraloaded*, 29 June. Available at: https://sierraloaded.sl/news/government-reduces-fuel-prices-nle33-nle35/ (Accessed: 16 October 2026).

Sierraloaded (2026b) 'Government announces new public transport fare following fuel price reductions', *Sierraloaded*, 1 July. Available at: https://sierraloaded.sl/news/government-public-transport-fare-fuel-eductions/ (Accessed: 16 October 2026).

State House Sierra Leone (2019) *President Julius Maada Bio launches urban mobility project, highlights cost of Freetown's congestion*, 28 November. Available at: https://statehouse.gov.sl/president-julius-maada-bio-launches-urban-mobility-project-highlights-cost-of-freetowns-congestion (Accessed: 16 October 2026).

State House Sierra Leone (2024) *Sierra Leone's President Julius Maada Bio commissions transit transport terminal and traffic signals to transform urban mobility*, 27 November. Available at: https://statehouse.gov.sl/sierra-leones-president-julius-maada-bio-commissions-transit-transport-terminal-and-traffic-signals-to-transform-urban-mobility (Accessed: 16 October 2026).

United Nations (2015) *Transforming our World: the 2030 Agenda for Sustainable Development* (A/RES/70/1). New York: United Nations. Available at: https://sdgs.un.org/2030agenda (Accessed: 16 October 2026).

World Bank (2024) 'From teetering to thriving: Transforming public transportation in Sierra Leone', *World Bank Blogs*, 3 June. Available at: https://blogs.worldbank.org/en/transport/from-teetering-to-thriving--transforming-public-transportation-i (Accessed: 16 October 2026).